DELIVERABLE 3 • SYSTEM ARCHITECTURE & ENGINEERING RATIONALE

SalesPulse BI: Architecture & Technical Decisions

Next.js 16 (React 19, TypeScript) • Express 5 (ES Modules) • MongoDB (Mongoose 9) • BullMQ 6 on Redis • Socket.io 4

1.1 System Overview: Decoupling Write and Read Workloads

SalesPulse BI is architected around separating two fundamentally different workloads. The **write path** (ingesting a 50 MB CSV or Excel export containing 100,000+ order lines) is slow, CPU/IO-intensive, and bursty; executing it inside an HTTP request cycle would block the Node.js event loop and trigger gateway timeouts. It is therefore offloaded to a background **BullMQ worker backed by Redis**, while the API responds immediately with `HTTP 202 Accepted` and streams live progress over **Socket.io**.

Conversely, the **read path** (interactive dashboard analytics across KPIs, time-series trends, dimensional breakdowns, and sales forecasting) consists of repeatable queries over immutable completed batches. It is served by single-pass **MongoDB aggregation pipelines** aligned with a single compound index and fronted by an **in-process TTL cache with request coalescing**.

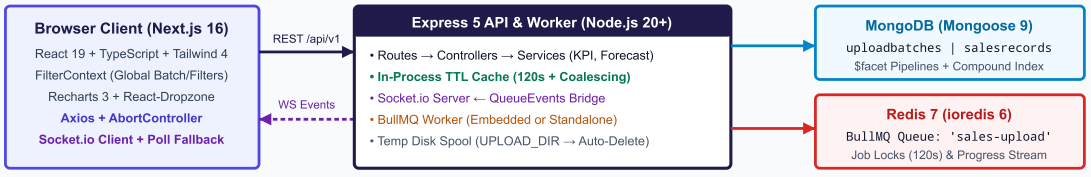


Figure 1.1: End-to-end architecture separating the BullMQ + Redis write pipeline from the cached MongoDB aggregation read path.

1.2 Monorepo Structure & Server Layering

- **Client Package (`client/`):** Built on **Next.js 16 (App Router)**, **React 19**, **TypeScript**, **Tailwind CSS 4**, and **Recharts 3**. Organized into `app/` (`layout.tsx` providers and `page.tsx` dashboard), `api/` (`axiosInstance.ts`, defensive response adapters in `analytics.api.ts`, and `socket.ts`), `components/` (`charts/`, `kpi/`, `layout/`, `tables/`, `upload/`), `context/FilterContext.tsx`, and shared contracts in `types/dashboard.types.ts`.
- **Server Package (`server/src/`):** Strictly layered **Express 5** application in ES Modules:
  - **Routes (`routes/api.routes.js`):** Maps endpoints to controllers and attaches rate limiters and Multer upload middleware.
  - **Controllers (`upload.controller.js`, `analytics.controller.js`):** Handles HTTP concerns exclusively—validating query parameters, checking the TTL cache, invoking services, and shaping JSON responses.
  - **Services (`fileParser.service.js`, `kpi.service.js`, `forecast.service.js`):** Pure domain and analytical logic with zero knowledge of HTTP request/response objects.
  - **Unit-Tested Pure Functions (`server/test/`):** Pipeline builders and mathematical helpers (`buildMatch`, `computeRevenueGrowth`, `shapeBreakdowns`, `calculateForecast`, `dateOrder`, `headerMapper`, `cache`) are exported separately and verified across 8 test suites using Node's built-in runner (`node --test`).

2.1 The Write Path: 7-Stage Asynchronous Upload Pipeline

1. **Multipart Stream & Magic-Byte Inspection:** Multer streams the uploaded file to `UPLOAD_DIR` under a random filename. `verifyUploadedFile` inspects the initial bytes to confirm `.xlsx` files are valid ZIP archives and `.csv` files contain text with no binary bytes.
2. **Batch Creation & Bounded Enqueue (`HTTP 202`):** The controller creates an `UploadBatch` with `status: 'queued'` , enqueues a `process-upload` job in BullMQ (bounded by `ENQUEUE_TIMEOUT_MS = 5000`), and returns `202 Accepted` immediately.
3. **Socket Room Subscription & Join Replay:** The browser emits `join:batch` . The server joins the socket to that batch's room and immediately replays the batch's current state from MongoDB so a small file that finished before subscription is never missed. The UI also polls `GET /uploads/:batchId` as a fallback.
4. **Pre-Clean, Header Mapping & Date Detection:** The worker marks the batch `processing` , deletes any rows from prior attempts, normalizes headers, verifies required columns upfront, and scans up to 2,000 rows to detect `dmy` vs. `mdy` date order.
5. **1,000-Row Chunked Validation & Bulk Insert:** Rows are validated in chunks of 1,000 using Zod (`salesRow.schema.js`). Valid rows are inserted via `insertMany(chunk, { ordered: false, lean: true })` ; invalid rows are appended to the quarantine log. Progress is emitted via Redis after every chunk and persisted to MongoDB at most once per second.
6. **Completion or Atomic Rollback:** On success, the batch becomes `completed` with final counts, `errorLogs` , `dateOrder` , and `notes` . On failure, the worker deletes all records inserted for that `batchId` and marks the batch `failed` . Temp disk files are always deleted.
7. **Dashboard Auto-Refresh:** On receiving `upload:completed` , the browser resets filters, refreshes the dataset dropdown, selects the new batch, and displays a data-health banner with a downloadable CSV quarantine report.

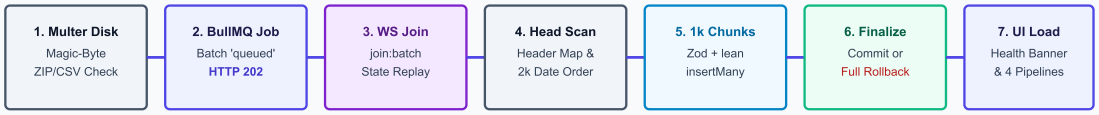


Figure 2.1: Step-by-step execution flow across the upload controller, BullMQ worker, Redis event bridge, and React UI.

2.2 The Read Path & Deterministic Forecasting Engine

Whenever the active batch or any filter changes in `FilterContext.tsx` , `page.tsx` triggers all four analytics endpoints (`overview` , `trends` , `breakdowns` , `forecast`) concurrently via `Promise.all` and passes an `AbortController` signal so stale in-flight requests are cancelled immediately when a user toggles filters rapidly. On the backend, `overview` and `breakdowns` use MongoDB `$facet` stages to compute multiple aggregations in a single pass over the compound index.

> **Forecasting Model (`forecast.service.js`):** Historical monthly revenue (with empty intermediate months zero-filled) forms chronological pairs $(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})$. When $n \geq 3$ months, the engine computes Ordinary Least Squares (OLS) linear regression slope $\mathbf{m = [n \sum(x \cdot y) - (\sum x)(\sum y)] \div [n \sum(x^2) - (\sum x)^2]}$ and intercept $\mathbf{b = (\sum y - m \sum x) \div n}$, then blends **70% OLS Linear Trend + 30% 3-Month Weighted Moving Average (WMA)** alongside R^2 fit quality and a 95% prediction interval (`lower` , `upper`). Under 3 months of history, it falls back to a Simple Moving Average (`sma`).

3.1 Architectural Decisions & Trade-Offs Matrix

Table 3.1 documents the 16 primary engineering decisions verified in the repository source code, why each choice was made, and its operational trade-off.

Architectural Decision	Engineering Rationale (Why)	Operational Trade-Off
1. Queue-based ingestion (BullMQ on Redis)	Parsing and inserting 100,000+ rows inside an HTTP request risks timeouts. The API answers `202` in milliseconds while the user watches live progress.	Redis becomes a mandatory runtime dependency Checked at boot (server refuses to start if Redis is unreachable).
2. Embedded or standalone worker (`RUN_WORKER_IN_API`)	Running inside the API process (`true`) is simplest for local dev and small hosts; running `npm run worker` separately isolates API latency on large files.	A standalone worker must share the same `UPLOAD_DIR` filesystem as the API (or transition to object storage across hosts).
3. Disk spooling (`UPLOAD_DIR`) with path in job	Streaming uploads to disk and passing only the file path keeps 50 MB files out of Node.js heap memory and out of Redis job payloads.	Requires shared disk access and automatic temp-file cleanup after each job and at server startup (for files > 24 hours old).
4. Streamed CSV & 1,000-row unordered `lean` inserts	Maintains flat memory usage on large CSVs. Unordered 1,000-row `insertMany` with `lean: true` skips Mongoose hydration after Zod has already typed every row.	Excel (`.xlsx`) files must still be loaded whole into memory due to format and library constraints.
5. Zod row validation with quarantine report	A single malformed row never discards a 50 MB upload. Every problem in a row is reported together with spreadsheet row number and column.	Users must review skipped rows; stored logs in MongoDB are capped at `MAX_ERROR_LOGS` (1,000) to protect BSON document size.
6. Per-file date-order detection (first 2,000 rows)	`03/04/2026` means different dates across regions; scanning for a day/month > 12 prevents silent date corruption.	Files without disambiguating dates fall back to `DEFAULT_DATE_ORDER` (`dmy`), surfaced to the user via batch `notes`.
7. No auto-retry (`attempts: 1`) + delete-before-insert	Prevents partial retries from duplicating rows and guarantees stalled-job recovery starts from a clean slate.	If a job fails permanently, the user must re-upload the file.
8. Two-minute job lock (`WORKER_LOCK_MS = 120000`)	Rides out brief Redis network drops without BullMQ prematurely restarting a healthy running ingestion job.	If a worker process crashes hard, its stalled job takes ~2 minutes to be recovered.
9. Single flat collection + one compound index	Every query filters by `batchId`; one compound index covers batch, date, category, and region filters while keeping bulk inserts fast.	Customer and product names repeat across rows; ad-hoc queries without `batchId` would scan the collection.
10. Compute metrics inside MongoDB pipelines	Aggregations run next to the data and count distinct orders/customers via grouping without pulling raw rows into Node.js.	Aggregation pipelines are more verbose than JS loops, mitigated by unit tests on the pipeline builders.
11. Explainable forecast (70% OLS + 30% 3-month WMA)	Deterministic, zero Python/ML runtime dependencies, easy to explain to business users, with R^2 and a 95% prediction range.	Does not model multi-year seasonality, promotions, or external macroeconomic factors.
12. In-process TTL cache + request coalescing	Completed batches are immutable; caching for 120s and coalescing concurrent identical queries eliminates duplicate DB work.	Cache lives inside a single Node.js process, so multiple API replicas maintain separate caches.
13. Redis `QueueEvents` as WebSocket bridge	The Express API remains the sole WebSocket endpoint whether the worker is embedded or remote; join state replay prevents race conditions.	Events broadcast to all browsers by default unless `SOCKET_ROOMS=true` is enabled.
14. React Context (`FilterContext`) as state store	Centralizes active dataset, filters, modals, and health banners (persisting `batchId` in `localStorage`) without Redux boilerplate.	Context consumers re-render on context changes (mitigated via memoization).
15. Thin client API layer with defensive adapters	`analytics.api.ts` centralizes a 60s timeout, comma-separated array serialization, and response shape normalization.	Accepting multiple field aliases can mask naming drift between frontend and backend contracts.
16. Express 5 with centralized	[illegible]	[illegible]

4.1 Security Hardening, Reliability & Operations

- **Query & Input Sanitization:** `batchId` parameters are validated against a 24-hex ObjectId regex before any database call. Filter arrays are capped at 100 items, and object-style query parameters (such as `category[a]=b`) are rejected to prevent NoSQL operator injection. Dates are parsed strictly and forecast `horizon` is bounded to `1–12`.
- **Upload & HTTP Hardening:** Enforces extension and MIME allowlists, first-byte magic header checks, random temporary filenames, automatic temp-file deletion, `helmet` security headers, CORS origin allowlisting, a 1 MB JSON body limit, response `compression`, and proxy-aware rate limiting.
- **Operational Resilience:** Fails fast at boot if MongoDB or Redis is unreachable; bounds Redis enqueue waits with `ENQUEUE_TIMEOUT_MS` (returning `503` instead of hanging); throttles repeated Redis outage logs to once every 30s; and executes graceful shutdown on `SIGINT` / `SIGTERM` (with a 10s forced exit fallback).

4.2 Key Runtime Configuration Reference (`server/.env` & `client/.env.local`)

Environment Variable	Default Value	Operational Purpose
`MONGODB_URI`	*Required (None)*	MongoDB connection string (pool size controlled by `MONGODB_MAX_POOL_SIZE`, default `20`).
`REDIS_URL`	`redis://127.0.0.1:6379`	Redis connection string (use `rediss://` for TLS; TCP keep-alive controlled by `REDIS_KEEPALIVE_MS = 10000`).
`RUN_WORKER_IN_API`	`true`	Runs the BullMQ worker inside the API process. Set `false` and run `npm run worker` separately for 100k+ row files.
`WORKER_CONCURRENCY` / `WORKER_LOCK_MS`	`1` / `120000`	Parallel upload jobs per worker and lock duration (ms) before BullMQ considers a silent job stalled.
`ENQUEUE_TIMEOUT_MS`	`5000`	Maximum time `POST /uploads` waits for Redis before rolling back and answering `503`.
`MAX_UPLOAD_MB` / `MAX_ERROR_LOGS`	`50` / `1000`	Maximum uploaded file size (MB) and maximum skipped-row entries stored inside `UploadBatch.errorLogs`.
`DEFAULT_DATE_ORDER`	`dmy`	Fallback order (`dmy` or `mdy`) for ambiguous `a/b/YYYY` dates when the first 2,000 rows contain no proof.
`STALE_UPLOAD_MINUTES`	`60`	Age threshold after which a stuck `queued` or `processing` batch can be deleted via `DELETE /uploads/:batchId`.
`CACHE_TTL_SECONDS` / `CACHE_MAX_ENTRIES`	`120` / `500`	Lifetime and capacity of the in-memory analytics cache for completed batches (`0` disables caching).
`SYNC_INDEXES`	`false`	Set `true` for one startup on an existing database to create missing indexes and drop obsolete single-field indexes.
`NEXT_PUBLIC_API_URL` / `NEXT_PUBLIC_SOCKET_URL`	`localhost:5000/api/v1` / `localhost:5000`	Client REST and Socket.io base URLs in `client/.env.local` (inlined by Next.js at build time).

4.3 Known Limitations & Production Roadmap

- **Authentication & Multi-Tenancy:** Currently, every visitor can view or delete any batch. Add JWT/session authentication and an `owner` field on `UploadBatch` before shared deployment.
- **Client/Server Minor Contract Drifts to Align:** (1) The client dropzone accepts `.xls`, whereas the server only accepts `.csv` and `.xlsx` (returning `400` for `.xls`). (2) The client's `BatchStatus` type uses `pending` while the server uses `queued`. (3) The client recomputes forecast confidence labels at R^2 thresholds of `0.75` / `0.45` (`Moderate`) whereas the server uses `0.70` / `0.40` (`Medium`). (4) The server returns 95% `lower` and `upper`